Performance Comparison:

	Logistic Regression	Support Vector Machine
Accuracy	79.33%	80.45%
Recall	69.57%	66.67%
Precision	75.00%	79.31%

- SVM has higher Accuracy and Precision, so we can choose SVM if we prioritize Overall Accuracy & Low False Positives
- Logistic Regression has higher Recall, so we can choose Logistic Regression if we prioritize Identifying Positives
- Both models are equally good

Logistic Regression:

```
# Classification Report
print(classification_report(y_test, y_pred, target_names=["Died (0)", "Survived (1)"]))
```

✓ 0.0s

```
              precision    recall  f1-score   support

    Died (0)       0.82      0.85      0.84       110
Survived (1)       0.75      0.70      0.72        69

    accuracy                           0.79       179
   macro avg       0.78      0.78      0.78       179
weighted avg       0.79      0.79      0.79       179
```

```
# Print Confusion Matrix Breakdown
tn, fp, fn, tp = confusion_matrix.ravel()
print(f"True Negatives: {tn}")
print(f"False Positives: {fp}")
print(f"False Negatives: {fn}")
print(f"True Positives: {tp}\n")
```

✓ 0.0s

```
True Negatives: 94
False Positives: 16
False Negatives: 21
True Positives: 48
```

Create a DataFrame containing three records (three passengers). Use the model to predict survival, and print the results:

```
new_passengers = [
    [3, 1, 36.0, 67.0],  # Passenger 1: 3rd Class Male, Age 36, Fare $67.00
    [3, 0, 67.0, 69.0],  # Passenger 2: 3rd Class Female, Age 67, Fare $69.00
    [2, 0, 69.0, 36.0]   # Passenger 3: 2nd Class Female, Age 69, Fare $36.00
]

# Convert to NumPy array
X_new = np.array(new_passengers)

# Predict
predictions = clf.predict(X_new)
probabilities = clf.predict_proba(X_new)

# Result
for i in range(len(predictions)):
    outcome = "Survived" if predictions[i] == 1 else "Died"
    surv_prob = probabilities[i][1] * 100

    print(f"Passenger {i+1}: {outcome} ({surv_prob:.1f}%)")
```

[11] ✓ 0.0s

```
Passenger 1: Died (7.0%)
Passenger 2: Died (26.2%)
Passenger 3: Survived (50.8%)
```

Support Vector Machine (SVM):

```
# Classification Report
print(classification_report(y_test, y_pred, target_names=["Died (0)", "Survived (1)"]))
```

✓ 0.0s

```
              precision    recall  f1-score   support

    Died (0)       0.81      0.89      0.85       110
Survived (1)       0.79      0.67      0.72        69

    accuracy                           0.80       179
   macro avg       0.80      0.78      0.79       179
weighted avg       0.80      0.80      0.80       179
```

```
# Print Confusion Matrix Breakdown
tn, fp, fn, tp = confusion_matrix.ravel()
print(f"True Negatives: {tn}")
print(f"False Positives: {fp}")
print(f"False Negatives: {fn}")
print(f"True Positives: {tp}\n")
```

✓ 0.0s

```
True Negatives: 98
False Positives: 12
False Negatives: 23
True Positives: 46
```

Create a DataFrame containing three records (three passengers). Use the model to predict survival, and print the results:

```
new_passengers = [
    [3, 1, 36.0, 67.0],  # Passenger 1: 3rd Class Male, Age 36, Fare $67.00
    [3, 0, 67.0, 69.0],  # Passenger 2: 3rd Class Female, Age 67, Fare $69.00
    [2, 0, 69.0, 36.0]   # Passenger 3: 2nd Class Female, Age 69, Fare $36.00
]

# Convert to NumPy array
X_new = np.array(new_passengers)

# Scale
scaled_passengers = scaler.transform(X_new)

# Predict
predictions = svm_clf.predict(scaled_passengers)

for i in range(len(predictions)):
    outcome = "Survived" if predictions[i] == 1 else "Died"
    print(f"Passenger {i+1}: {outcome}")

# SVM optimize geometric margin rather than sigmoid activations
```

[12] ✓ 0.0s

```
Passenger 1: Died
Passenger 2: Died
Passenger 3: Survived
```